

HAETAE-2 Jasmin 구현의 핵심 연산 기능정확성 검증 목표

KeyGen 키 방정식, Sign 응답·힌트, Verify 재구성·승인

HAETAE formal verification project

2026년 8월 11일

Abstract

이 문서는 HAETAE-2 Jasmin 구현에 대해 최종적으로 증명할 기능정확성 명제를 고정한다. 대상은 전체 확률분포나 EUF-CMA 보안이 아니라, 실제 추출된 KeyGen, Sign, Verify 절차 안에서 HAETAE 고유의 계산을 수행하는 핵심 경로이다. KeyGen에서는 저장된 검증키와 비밀키가 만드는 키 방정식, Sign에서는 challenge, response, hint 및 두 rejection predicate, Verify에서는 서명 성분의 복원, 공개키 행렬식, challenge 재계산 및 norm 판정을 검증한다. 마지막으로 세 결과를 합성하여 accepted Sign 계산이 Verify 핵심 predicate를 만족함을 보인다. 모든 명제는 실제 프로시저, mode-2 상수, 종료성 양식과 byte/object 경계를 명시한다. 또한 각 대상이 HAETAE correctness의 어느 전제를 구현하는지, 구현 오류가 어떤 false reject·명세 밖 승인·표현 단절을 만드는지, 해당 정리가 그 실패를 어떻게 배제하는지를 명시한다. 검증 방법은 pinned Jasmin 추출, 수학 object와 word/array 표현 관계, 실제 procedure Hoare 정리, 호출 순서 합성 및 fresh proof audit의 단계로 재현 가능하게 고정한다.

최종 보고 문장.

HAETAE-2 $(q, n, k, \ell) = (64513, 256, 2, 4)$ Jasmin 구현에서 KeyGen의 키 방정식 계산, Sign의 challenge·response·hint·거부조건 계산, Verify의 서명 재구성·challenge 재계산·norm 판정을 EasyCrypt로 기계검증하고, accepted KeyGen과 Sign의 결과가 Verify 핵심 계산에서 승인됨을 증명한다.

상태 주의.

이 문서는 목표 정리(theorem target)를 명세한다. 이 문서에 수식이 적혀 있다는 사실은 해당 정리가 이미 컴파일되었다는 뜻이 아니다. 실제 상태는 claim ledger와 fresh EasyCrypt 검증 로그로만 판정한다.

현재 실행 범위는 **GO-MINCORE-7D**이다. 7일 동안 KeyGen, Sign, Verify의 focused actual core 정리와 decoded-object 제한 합성을 목표로 하며, h codec, full parser, termination, 분포 및 public API 전체 정리는 보류한다.

목차

1	검증 당위성과 실패 모델	3
1.1	왜 명세의 정확성 정리만으로 충분하지 않은가	3
1.2	고려하는 구현 실패	3
1.3	각 검증 대상의 필요성	4
1.4	현재 범위가 정당화하는 결론	5

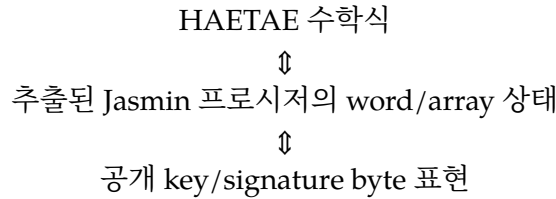
2	검증 방법론	5
2.1	재현 가능한 일곱 단계	5
2.2	소스 고정, 추출과 증명 경계	6
2.3	수학 명세와 구현 표현의 연결	6
2.4	국소 Hoare 정리의 표준 형태	7
2.5	대상별 증명 전략	7
2.6	제어흐름, 종료성과 non-vacuity	8
2.7	합성 정리와 완료 증거	8
3	고정 범위와 구현 경계	9
3.1	HAETAЕ-2 파라미터	9
3.2	실제 Jasmin 프로시저	9
3.3	정확성 양식	10
4	KeyGen: 핵심 상태가 만족해야 하는 정확한 키 방정식	10
4.1	중간값과 저장 표현	11
4.2	키 방정식	11
5	Sign: challenge, response, hint 및 rejection	12
5.1	입력 경계	12
5.2	실제 계산과 대응할 식	12
6	Verify: 복원식과 최종 승인 predicate	13
6.1	파싱 이후의 signature object	13
6.2	승인 조건	13
7	KeyGen-Sign-Verify 제한 합성	14
7.1	합성할 세 사실	14
8	아티팩트 산출물과 완료 판정	15
8.1	작업 순서	15
8.2	각 headline theorem의 완료 기준	15
8.3	안전성 및 TCB	16
8.4	명시적 비주장	16
8.5	예상 작업량과 보고 제목	16

1 검증 당위성과 실패 모델

1.1 왜 명세의 정확성 정리만으로 충분하지 않은가

HAETAE 명세가 요구하는 correctness는 KeyGen이 만든 키쌍과 메시지에 대해 Sign이 만든 서명을 Verify가 승인하는 성질이다 [2]. 명세의 correctness 및 보안 정리는 KeyGen, Sign, Verify가 명세에 적힌 환 연산, 분해, 해시 transcript, norm 판정을 그대로 수행한다는 것을 전제로 한다. 그러나 실제 Jasmin 구현에는 NTT/CRT 배열, 고정폭 word, 포인터와 offset, in-place 갱신, packing 및 rejection branch가 들어간다. 따라서 명세 정리가 참이라는 사실만으로 추출된 프로시저가 그 정리의 알고리즘을 구현한다는 결론은 나오지 않는다.

본 작업이 닫으려는 공백은 다음 세 층 사이의 대응이다.



첫 번째 화살표가 없으면 구현이 다른 수식을 계산해도 “명세가 안전하다”는 사실만 남는다. 두 번째 화살표가 없으면 object 수준에서 올바른 값이 실제 공개 API의 byte 배열에 보존되는지 알 수 없다. 이 문서의 핵심 정리는 첫 번째 화살표와 두 번째 화살표의 현재 증명 가능한 부분을 실제 추출 코드에 대해 확립한다. 이는 구현의 기능정확성이며, 그 자체로 sampler 분포, side channel, termination 또는 EUF-CMA를 증명하지는 않는다.

1.2 고려하는 구현 실패

유한한 test vector는 실행한 입력에서 나온 값만 비교한다. 본 검증은 다음과 같이 입력 공간 전체에 걸쳐 나타날 수 있는 구조적 실패를 정리의 반례로 만든다.

계산 불일치

부호, modulus, rounding, norm bound 또는 rejection 조건이 명세와 다른 경우.

표현 불일치

coefficient 순서, NTT/CRT 표현, signed/unsigned word, byte offset 또는 pack/unpack 변환이 같은 수학 object를 나타내지 않는 경우.

제어흐름 불일치

실패 뒤의 상태를 성공으로 사용하거나, 성공해야 하는 후속 절차를 건너뛰거나, theorem이 실제로 도달할 수 없는 성공 branch만 기술하는 경우.

경계 불일치

개별 프로시저의 정리는 맞지만 key, transcript, signature object의 해석이 호출자와 피호출자 사이에서 달라 합성되지 않는 경우.

실행 의미의 공백

추출 모델의 postcondition은 맞지만 memory safety가 없어 그 결과를 compiled Jasmin 실행으로 올릴 수 없는 경우.

어떤 오류가 실제 위조로 이어진다고 이 문서가 주장하지는 않는다. 다만 아래 계산이 명세와 다르면 구현이 명세의 correctness 정리 또는 보안 환원의 전제를 만족한다고 말할 근거가 사라진다. 따라서 보안 증명과 별개로 이 대응을 검증해야 한다.

1.3 각 검증 대상의 필요성

대상은 임의로 선택한 helper 목록이 아니라, 키 생성에서 최종 승인까지 값이 흐르는 한 개의 의존 사슬을 따른다.

KeyGen 키 관계 $\rightarrow$ Sign의 commitment와 response $\rightarrow$ hint/codec
 $\rightarrow$ Verify의 복원 $\rightarrow$ challenge $\cdot$ norm 승인.

대상	HAETAE에서의 역할	구현 오류가 만드는 구체적 실패	정리가 제공하는 보증
KeyGen	검증키와 비밀키가 같은 키쌍임을 나타내는 $As = qj \pmod{2q}$ 를 만들고, 공개되는 b_1 과 비밀 성분을 저장한다.	잘못된 truncation, NTT 곱 또는 부호는 Sign이 사용하는 비밀 성분과 Verify가 재생성할 공개 행렬의 관계를 끊는다.	actual matrix/finalization core의 배열이 (KG-1)–(KG-4)와 $As = qj \pmod{2q}$ 를 만족함을 보인다. byte packing과 parent guard는 별도 확장이다.
Sign challenge	$(w_1, \text{LSB}(y_0), \mu)$ 를 hash하여 commitment, 메시지와 challenge를 결합한다.	byte transcript, component 순서 또는 challenge weight가 다르면 Sign과 Verify가 서로 다른 challenge를 계산한다.	실제 transcript와 (S-3)의 challenge 식이 정확히 대응함을 보인다.
Sign response · rejection	$z = y + (-1)^b cs$ 를 만들고 명세의 두 norm 조건을 통과한 trial만 반환한다.	곱셈 · 부호 오류는 challenge에 대한 올바른 response 관계를 깨뜨린다. 비교 연산, strictness 또는 상수 오류는 명세 밖의 trial을 승인하거나 유효 trial을 버린다.	actual z 식과 $\text{reject} = 0$ 의 두 정수 norm predicate를 동치로 연결한다.
Sign hint	전송하지 않은 z_2 정보를 대신하여 Verify가 Sign과 같은 w_1 을 복원하게 한다.	high-bit 경계, centered reduction 또는 modulus 오류는 올바른 서명의 challenge 재계산을 다르게 만들어 false reject를 일으킨다.	실제 coefficient마다 명세의 hint 식이 성립하고 Verify 복원 lemma에 사용할 범위가 보존됨을 보인다.
Verify reconstruction	압축된 (x, v, h, c) 와 VK에서 $\tilde{z}_1, w', u, w_1, \tilde{z}_2$ 를 재구성한다.	parity, division by two, arithmetic shift 또는 centered reduction 오류는 공격자가 제공한 byte가 의도한 수학 object와 다른 값으로 해석되게 한다.	실제 word/array 계산을 (V-1)–(V-6)의 integer/ring 식으로 refine한다.
Verify acceptance	재구성된 norm과 message-bound challenge를 검사하는 외부 승인 경계이다.	잘못된 boolean 조합은 honest signature를 거부하거나, 명세 predicate를 만족하지 않는 object를 승인한다. 후자는 곧바로 위조 증명은 아니지만 명세 보안 정리의 구현 적용을 무효화한다.	actual $\text{reject} = 0$ 과 $\text{norm} \cdot \text{challenge}$ 조건의 필요충분 동치를 보인다.

대상	HAETAЕ에서의 역할	구현 오류가 만드는 구체적 실패	정리가 제공하는 보증
HBZ/rANS codec (기존 byte 경계)	Sign의 z_1 high coefficients와 실제 signature byte 구간을 연결한다.	순서·길이·offset 오류는 다른 object를 encode하거나 decode한다. 성공 branch가 도달 가능하다는 증인 없이 inverse만 증명하면 정리가 공허할 수도 있다.	actual encoder/copy/decoder inverse와 frame 보존을 제공한다. fixed all-6 결과는 종료한 실행의 실패를 배제하지만 termination/non-vacuity는 제공하지 않는다.
제한 합성	KeyGen, Sign, decoded signature object, Verify의 국소 정리를 하나의 honest accepted execution으로 연결한다.	각 정리가 서로 다른 key 배열, message transcript 또는 representation을 가정하면 모든 국소 정리가 참이어도 최종 Verify는 거부할 수 있다.	동일 key/message와 object identity 아래 actual Verify core의 $reject = 0$ 을 도출한다.

1.4 현재 범위가 정당화하는 결론

활성 MINCORE 대상들은 accepted KeyGen과 Sign 실행이 만든 수학 object가 동일한 key와 message transcript 아래 Verify core에서 승인된다는 **제한된 implementation completeness**를 주장하기 위해 모두 필요하다. 하나라도 빠지면 다음과 같은 단절이 남는다.

- KeyGen 정리가 없으면 Verify 재구성에서 사용하는 키 관계를 구현에서 얻지 못한다.
- Sign 또는 Verify 정리가 없으면 같은 challenge와 norm predicate를 계산했다는 사실을 얻지 못한다.
- full codec 정리가 없으면 public signature byte 배열까지 결론을 올릴 수 없지만, 명시적 decoded-object identity를 둔 현재 합성은 수행할 수 있다.
- 합성 정리가 없으면 개별 postcondition의 전제가 실제 호출 사이에서 동시에 성립한다는 보장이 없다.
- safety 증거가 없으면 결론은 extracted EasyCrypt model에 머문다.

반대로 이 대상들만으로는 임의 byte 입력에 대한 parser soundness, sampler의 확률분포, rejection loop의 termination, constant-time 실행 또는 EUF-CMA를 결론낼 수 없다. 이 구분 때문에 본 작업은 전체 보안 증명보다 먼저 결정적 핵심 경로와 표현 경계를 닫고, 제외한 성질은 독립된 후속 정리로 남긴다.

2 검증 방법론

2.1 재현 가능한 일곱 단계

본 작업에서 “EasyCrypt로 검증한다”는 말은 수학식을 별도 모델에서 다시 계산한다는 뜻이 아니다. pinned Jasmin 구현에서 추출된 실제 procedure와 HAETAЕ 명세의 수학 object 사이에 refinement 정리를 세우고, 그 정리들을 실제 호출 순서대로 합성한다는 뜻이다. 전체 절차는 다음 일곱 단계로 고정한다.

1. **소스와 추출 경계 고정.** Jasmin source hash, mode 2 호출 상수와 대상 procedure 목록을 고정하고 jasmin2ec로 EasyCrypt procedure를 생성한다.

2. **수학 명세 고정.** HAETAE 명세의 환·정수 식, decomposition, norm predicate와 byte/object 경계를 실행 코드와 독립적으로 정의한다.
3. **표현 관계 정의.** 수학 polynomial·vector·signature object가 구현의 NTT/CRT 배열, 고정폭 word와 byte slice에 어떻게 표현되는지 predicate로 명시한다.
4. **실제 procedure의 국소 Hoare 정리.** 추출된 procedure를 직접 호출하여 precondition에서 실제 postcondition을 도출한다. 성공이나 원하는 출력 equality를 전제로 넣지 않는다.
5. **제어흐름과 정확성 양식 분리.** loop invariant, failure/success branch, frame을 증명하고 partial correctness, losslessness, termination을 서로 다른 주장으로 관리한다.
6. **호출 순서 합성.** KeyGen, Sign, Verify의 국소 정리를 실제 caller/callee binding, 동일 transcript와 decoded-signature-object identity 아래 합성하여 제한된 최종 승인 정리를 만든다.
7. **기계적 검증과 증거 보존.** fresh compile, proof-hole·axiom scan, extraction drift, safety evidence와 claim ledger를 검사하고 재현 가능한 로그를 남긴다.

이 순서에서 앞 단계는 뒤 단계의 theorem precondition을 정당화한다. 앞 단계가 미완료이면 그 사실을 새 가정으로 숨기지 않고 최종 claim의 경계로 공개한다.

현재 7일 MINCORE 실행에서는 decoded signature object를 합성 경계로 고정한다. HBZ/rANS의 기존 결과는 재사용 증거이지만, h , metadata, padding, full parser를 새로 달는 작업은 보류한다. 따라서 byte codec 방법은 후속 확장의 방법론이지 이번 headline theorem의 완료 조건이 아니다.

2.2 소스 고정, 추출과 증명 경계

검증 단위는 함수 이름이 비슷한 새 observation model이 아니라 [section 3](#)에 열거한 pinned Jasmin procedure의 EasyCrypt 추출 결과이다. 각 최종 정리 또는 그 직접 하위 정리는 해당 generated procedure를 proc 본문에서 직접 호출해야 한다. proof-friendly wrapper를 사용할 경우에는 wrapper와 generated procedure 사이의 exact equivalence를 먼저 증명한다. 이 조건은 구현의 branch, word 연산 또는 memory update를 수학적식으로 대체한 뒤 그 대체 모델만 증명하는 것을 막는다.

추출은 다음 두 경계를 구분한다.

- **EasyCrypt 실행 경계:** generated procedure의 명령 의미에 대한 Hoare 결과이다.
- **compiled Jasmin 경계:** 위 결과에 Jasmin compiler theorem을 적용한 결과이며, 대상 procedure의 safety 증거를 추가로 요구한다.

따라서 safety가 확보되지 않은 정리는 “extracted EasyCrypt model 수준”으로만 표기하고 assembly-level functional correctness로 올리지 않는다. compiler, extraction, EasyCrypt kernel 및 representation bridge는 [section 8](#)의 TCB에 기록한다.

2.3 수학 명세와 구현 표현의 연결

명세는 R_q 의 polynomial·vector와 무한정수 연산으로 기술하지만 구현은 고정 길이 배열과 word를 사용한다. 각 procedure precondition에는 적어도 다음 세 층을 동시에 둔다.

$$\begin{array}{c}
 \text{Spec}(x) \\
 \Updownarrow \text{Rep}(x, a) \\
 \text{ArrayWordState}(a, m) \\
 \Updownarrow \text{Memory}(m) \\
 \text{ConcreteBytesAndPointers.}
 \end{array}$$

여기서 Rep는 결과 equality의 다른 이름이 아니라, 입력 배열이 어떤 수학 object를 나타내는지 정의하는 독립 predicate이다. 필요한 bridge lemma는 다음 범주로 나눈다.

- NTT/CRT 배열과 R_q 의 coefficientwise polynomial 연산,
- W64/W32 값과 정수 사이의 범위 및 no-wraparound 관계,
- signed/unsigned 변환, arithmetic shift와 division by two,
- HighBits, LowBits, LSB, centered reduction과 rounding,
- byte offset, slice equality, encode/decode와 coefficient 순서,
- pointer separation, 변경 구간과 보존해야 할 prefix · suffix frame.

상수 범위나 memory-disjointness가 필요한 경우 theorem statement에 공개한다. SMT가 우연히 concrete 상수를 풀었다는 사실을 representation theorem으로 대체하지 않는다.

2.4 국소 Hoare 정리의 표준 형태

각 actual procedure 정리는 다음 형태를 따른다.

```
{ ArgBinding ∧ Mode2Constants ∧ InputRep ∧ MemoryContract }
  Actual.M._procedure
{ ExactResult ∧ OutputRep ∧ BranchState ∧ Frame }.
```

ArgBinding은 harness 인자와 procedure formal argument의 동일성을, Mode2Constants는 실제 q, n, k, ℓ, τ 및 길이를, MemoryContract는 유효 범위와 alias 조건을 고정한다. postcondition은 수학 결과뿐 아니라 반환 code, 실행된 branch, offset, 변경하지 않은 배열 구간을 함께 기술한다.

다음 항목은 원하는 결론을 가정하는 것이므로 원칙적으로 precondition에 두지 않는다.

- encoder 또는 decoder가 성공했다는 가정,
- decoded object가 입력과 같다는 가정,
- copied trace나 decoder byte read가 encoder 출력과 같다는 가정,
- Sign 또는 Verify의 최종 reject 값,
- KeyGen의 최종 키 방정식이나 packing equality.

이 값들은 실제 하위 procedure의 postcondition과 bridge lemma에서 도출해야 한다. 도출할 수 없는 경우에는 theorem 이름과 claim 상태에 그 한계를 표시한다.

2.5 대상별 증명 전략

대상	실제 증명 경계	핵심 증명 방법
KeyGen	<code>_kp_m23_matrix</code> , <code>_keypair_finalize_m23</code> 의 focused actual sequence	matrix/NTT 결과를 (KG-1)로 refine하고 finalization에서 (KG-2)를 얻는다. 배열로 구성한 A, s 에 환 계산을 적용하여 $As = qj \pmod{2q}$ 를 도출하고 return/frame 상태를 보존한다. accepted parent의 guard, 길이와 packing lift는 7일 MINCORE 완료 조건에서 제외한다.

대상	실제 증명 경계	핵심 증명 방법
Sign	<code>_sf_round_challenge_mode2</code> , <code>_sf_z_check</code> , <code>_sf_hint_mode2</code> 의 순차 실행	raw μ memory transcript와 challenge hash 입력을 연결하고, NTT challenge product를 cs 로 refine한다. fixed-point accumulator와 정수 squared norm의 범위를 증명하여 두 rejection comparison을 동치화하고, high-bit bridge로 hint 식을 얻는다.
Verify	canonical object 이후 <code>_verify_prepare_z1_wprime</code> 부터 <code>_sign_verify_tail_m23</code> 까지	byte/high-low decomposition에서 $\tilde{z}_1, w'$ 를 복원하고, CRT matrix 결과를 u 로 refine한다. 분자의 parity와 word 범위를 이용해 arithmetic shift를 정수 나눗셈으로 연결한 뒤, 반환 $reject = 0$ 과 $norm \cdot challenge$ conjunction의 필요충분 동치를 증명한다.
HBZ/rANS codec (기존 증거)	actual encoder, suffix copy, actual decoder와 full wrapper; Week 16 확장 없음	encoder의 <code>segment_matches</code> , <code>copy</code> 의 <code>slice_eq</code> , decoder의 pointwise read 조건을 adapter lemma로 연결한다. failure/success postcondition과 frame을 모두 유지한다. fixed all-6 결과는 종료한 실행의 실패만 배제하며 termination/losslessness를 제공하지 않는다고 표기한다.
제한 합성	accepted KeyGen · Sign 결과와 actual Verify core	동일 VK, prefix/context, message memory로 $\mu = \tilde{\mu}$ 를 얻고 명시적 decoded-object identity로 signature object를 전달한다. KeyGen의 키 식과 Sign의 hint 식을 Verify 복원 lemma에 대입하여 최종 challenge equality와 norm 승인을 도출한다.

2.6 제어흐름, 종료성과 non-vacuity

KeyGen과 Sign에는 rejection loop가 있으므로 일반 headline theorem은 **종료한 accepted execution**에 대한 Hoare partial correctness로 작성한다. 이는 해당 실행이 종료하면 postcondition이 성립한다는 뜻이지, 모든 입력에서 loop가 종료한다거나 기대 반복 횟수가 유한하다는 뜻이 아니다. loop 본문에서는 accepted branch에 필요한 invariant와 rejected iteration 뒤의 frame을 증명하고, termination/losslessness는 별도 claim으로 관리한다.

codec처럼 실제 procedure가 실패할 수 있는 경우에는 결과를 다음처럼 분기한다.

- failure branch: 반환 code, 미실행 후속 절차와 보존된 상태,
- success branch: 정확한 크기 · trace · decoded equality와 frame.

success-conditioned partial correctness만으로는 성공 branch가 실제로 도달한다고 결론낼 수 없다. Week 15 결과는 all-zero HBZ의 종료한 실행이 성공함을 보이지만 termination/losslessness를 보이지 않으므로 non-vacuous witness로 부르지 않는다. Verify core는 고정 길이 loop의 bounded 실행으로 다루며, 전체 parser나 malformed-input 경로를 포함하지 않으면 그 사실을 precondition에 명시한다.

2.7 합성 정리와 완료 증거

국소 정리는 caller가 실제로 제공하는 배열과 memory를 precondition에 대입할 수 있을 때만 합성한다. composition harness는 production 호출 순서를 보존하고, procedure 사이에서 다음 상태를 명시적으로 전달한다.

- KeyGen이 만든 VK/SK object와 $As = qj \pmod{2q}$,
- Sign과 Verify가 공유하는 VK, prefix/context, message transcript,
- Sign의 (z, h, c) 와 동일 object를 나타내는 decoded (x, v, h, c) ,
- 각 호출의 pointer binding, 변경 구간과 frame.

최종 목표는 다음 implication을 actual procedure composition의 postcondition으로 도출하는 것이다.

$$\begin{aligned} & \text{TerminatedAcceptedKeyGen} \wedge \text{TerminatedAcceptedSign} \\ & \wedge \text{SameKeyAndMessageTranscript} \wedge \text{DecodedSignatureObjectIdentity} \\ & \implies \text{ActualVerifyCore.reject} = 0. \end{aligned}$$

정리는 EasyCrypt가 한 번 통과했다는 사실만으로 완료 처리하지 않는다. 각 신규 target의 fresh-no-eco compile, 전체 aggregate compile, proof-hole·authored axiom·debug declaration scan, extraction regeneration/hash 및 generated table drift, read-only source root, safety 결과와 독립 audit를 함께 확인한다. theorem manifest, claim ledger, 보고서와 논문이 같은 precondition과 claim status를 가질 때만 [section 8](#)의 완료 기준을 만족한다.

3 고정 범위와 구현 경계

왜 범위를 먼저 고정하는가. “HAETAE 구현을 검증했다”는 문장만으로는 parameter set, 호출 상수, 실제 procedure와 수학 모델 사이의 연결 여부를 판단할 수 없다. mode 2와 아래 actual 경계를 고정해야 정리가 어느 실행을 포괄하는지 재현할 수 있고, 다른 mode나 관찰용 wrapper에 대한 결과를 production 구현의 결과로 오인하지 않는다. 구체적인 추출·표현·Hoare 합성 절차는 [section 2](#)를 따른다.

3.1 HAETAE-2 파라미터

검증 대상은 HAETAE mode 2 하나이다. 환과 핵심 상수는 다음과 같이 고정한다 [2].

$$\begin{aligned} R_q &= \mathbb{Z}_q[X]/(X^{256} + 1), & q &= 64513, \\ n &= 256, & k &= 2, & \ell &= 4, & \tau &= 58, & \alpha_h &= 512, \\ m_h &= \frac{2(q-1)}{\alpha_h} = 252. \end{aligned}$$

여기서 $A_{\text{gen}} \in R_q^{2 \times 3}$, $a \in R_q^2$, $s_{\text{gen}} \in R_q^3$, $e_{\text{gen}} \in R_q^2$, 그리고 $j = (1, 0)^T \in R_q^2$ 이다.

3.2 실제 Jasmin 프로시저

목표 정리는 새로 만든 관찰용 모델만을 대상으로 하지 않는다. 최종 정리 또는 그 직접 하위 정리는 다음 pinned Jasmin 프로시저의 추출 결과를 호출해야 한다 [1].

Mode 2 상위 호출의 구현 상수는 다음과 같다.

KeyGen	$(k, m, vk , \tau) = (2, 3, 992, 58)$	singular bound 611098
Sign	$(sk , \sigma) = (1408, 1474)$	response bounds는 section 5 에 명시
Verify	$(k, \ell, m) = (2, 4, 3)$	$ \sigma = 1474, vk = 992$

영역	상위 경계	검증할 핵심 하위 절차
KeyGen	focused core harness	<code>_kp_m23_matrix</code> , <code>_keypair_finalize_m23</code> ; <code>_keypair_full_m23</code> 및 packer lift는 후속 범위
Sign	<code>_sf_signature_core_mode2</code>	<code>_sf_round_challenge_mode2</code> , <code>_sf_z_check</code> , <code>_sf_hint_mode2</code>
Verify	<code>_verify_full_m23</code>	<code>_verify_prepare_z1_wprime</code> , <code>_verify_matrix_crt</code> , <code>_sign_verify_recover_w_z2</code> , <code>_sign_verify_norm_reject</code> , <code>_sign_verify_tail_m23</code>
Codec (기존 증거)	<code>_encode_hb_z1_full</code> , <code>_decode_hb_z1_full</code>	actual rANS encoder, suffix copy, actual rANS decoder; Week 16 신규 확장 없음

표 3: 논문에서 직접 연결할 Jasmin 경계

3.3 정확성 양식

KeyGen과 Sign에는 rejection loop가 있다. 본 범위의 일반 정리는 다음 형식을 사용한다.

- **KeyGen/Sign:** 종료하여 accepted branch를 반환한 실행의 기능적 부분정확성 (partial correctness).
- **Verify:** canonical signature object와 유효한 메모리 계약 아래 bounded core 실행의 accept predicate 정확성.
- **고정 증인:** all-zero HBZ 같은 구체 입력은 별도의 losslessness/termination 정리가 있을 때만 non-vacuous witness로 부른다.

범위 경계 3.1 (Object 경계와 byte 경계). 주요 KeyGen–Sign–Verify 합성은 decoded polynomial/signature object 경계에서 수행한다. 현재 실제 HBZ/rANS byte codec 결과는 이 object 경계의 한 부분을 byte 배열까지 연결한다. 전체 1474-byte canonical parser를 주장하려면 별도로 h codec, metadata, padding 및 malformed-input rejection을 닫아야 한다.

Codec 검증의 당위성. 대수 정리가 올바른 signature object를 말하더라도 packer가 다른 coefficient 순서나 길이로 byte를 쓰면 public API에서 그 object는 전달되지 않는다. 반대로 decoder가 같은 byte를 다른 object로 읽으면 Verify 정리는 Sign 결과에 적용되지 않는다. 따라서 actual encode–copy–decode inverse는 단순 저장 형식 검사가 아니라 object 정리를 실제 서명 구간으로 운반하는 필수 refinement이다. 또한 success-conditioned inverse와 concrete success witness를 분리하여, 도달할 수 없는 성공 분기에 대해서만 성립하는 공허한 정리를 구분한다. 현재 fixed all-6 결과는 종료한 실행의 실패를 배제하지만 termination/losslessness를 증명하지 않으므로 non-vacuous 실행 증거로 승격하지 않는다. h 와 full parser는 7일 MINCORE 범위에서 명시적으로 보류한다.

4 KeyGen: 핵심 상태가 만족해야 하는 정확한 키 방정식

검증 당위성. HAETA의 bimodal signing과 Verify 재구성은 KeyGen이 만든 공개 행렬과 비밀 vector가 $As = qj \pmod{2q}$ 를 만족한다는 사실에 의존한다 [2]. 이 관계는 Sign과 Verify가 독립적으로 사용하는 두 byte 배열이 실제로 같은 키쌍임을 나타내는 연결 불변식이다. finalization의 부호, truncation 또는 NTT/CRT 곱이 틀리면 후속 packer와 각 절차가 정상 종료하더라도 이 불변식이 깨질 수 있다. 그러므로 출력 길이나 test-vector 일치가 아니라 실제 matrix/finalization

core가 만든 배열을 환의 키 식으로 해석하여 검증해야 한다. 992/1408-byte packing과 accepted parent lift는 이 core 정리와 구분한다.

4.1 중간값과 저장 표현

한 KeyGen iteration에서 sampler와 행렬 곱이 다음 값을 만든다고 하자.

$$b = a + A_{\text{gen}}s_{\text{gen}} + e_{\text{gen}} \pmod{q}. \quad (\text{KG-1})$$

실제 `_keypair_finalize_m23`는 각 coefficient에 대해 $b_0 \in \{-1, 0, 1\}$ 와 저장되는 high part b_1 을 계산하여

$$b = b_0 + 2b_1 \quad (\text{KG-2})$$

을 만족시켜야 한다. 여기서 b_1 은 992-byte verification key에 packing되는 값이다. 이 저장 표현을 기준으로 verifier가 사용하는 전체 키 행렬과 secret vector를 다음과 같이 정의한다.

$$A := (2(a - 2b_1) + qj \mid 2A_{\text{gen}} \mid 2I_2) \pmod{2q}, \quad (\text{KG-3})$$

$$s := (1 \mid s_{\text{gen}} \mid e_{\text{gen}} - b_0). \quad (\text{KG-4})$$

4.2 키 방정식

section 4의 핵심 결론은 모호한 “키가 올바르다”가 아니라 다음 coefficientwise 환 등식이다.

$$\begin{aligned} As &= 2(a - 2b_1) + qj + 2A_{\text{gen}}s_{\text{gen}} + 2(e_{\text{gen}} - b_0) \\ &= 2b - 4b_1 - 2b_0 + qj \\ &= qj \pmod{2q}. \end{aligned}$$

검증 목표 4.1 (KeyGen core theorem). 실제 `_kp_m23_matrix`와 `_keypair_finalize_m23`를 mode-2 $(k, m) = (2, 3)$ 배열 상태에서 호출한다. matrix 입력은 $a, A_{\text{gen}}, s_{\text{gen}}, e_{\text{gen}}$ 의 명시적 NTT/CRT representation을 만족해야 하며, desired key equation은 전제로 두지 않는다. 종료한 core 실행의 결과 배열을 decoding하여 다음 object를 얻는다.

$$(seed_A, b_1, s_{\text{gen}}, e_{\text{gen}} - b_0, K).$$

이 object가 (KG-1)–(KG-4)를 만족하고

$$As = qj \pmod{2q}$$

를 만족함을 증명한다. 반환 code, 실제 변경 구간과 비대상 array frame도 함께 보존한다.

이 목표에는 seed에서 sampler 출력으로 가는 확률분포의 정확성이나 rejection loop의 거의확실한 종료를 포함하지 않는다. NTT/CRT 배열 표현과 $A_{\text{gen}}s_{\text{gen}}$ 사이의 refinement는 포함한다. 실제 `_keypair_full_m23`의 guard, 992/1408-byte packing과 public API lift는 이번 MINCORE 완료 조건이 아니다.

5 Sign: challenge, response, hint 및 rejection

검증 당위성. Sign의 네 대상은 서로 독립적인 산술 helper가 아니다. challenge는 commitment와 message transcript를 묶고, response z 는 그 challenge와 비밀키의 관계를 구현하며, 두 rejection predicate는 명세가 반환하도록 허용한 norm 영역을 결정한다. hint는 전송하지 않은 z_2 대신 Verify가 같은 w_1 을 복원하게 한다 [2]. 따라서 transcript 순서, 부호, 비교의 strictness 또는 high-bit modulus 중 하나의 오류도 명세 밖 trial의 반환이나 honest signature의 거부를 만들 수 있다. 아래 정리는 이 네 의미가 실제 accepted 실행 하나에서 동시에 성립함을 보이는 데 필요하다. 다만 이는 sampler 분포나 rejection termination을 증명한다는 뜻은 아니다.

5.1 입력 경계

Sign core theorem은 다음을 입력 계약으로 사용한다.

- SK 배열은 [section 4](#)의 $(seed_A, b_1, s_{\text{gen}}, e_{\text{gen}} - b_0, K)$ 를 encoding 한다.
- $\mu = H_{\text{gen}}(vk[0..992], pre, M)$ 는 실제 raw/internal Sign transcript가 계산한 64-byte 값이다.
- hyperball sampler가 한 iteration의 (y, b, b') 를 제공한다. 이 장은 그 출력분포를 증명하지 않는다.

5.2 실제 계산과 대응할 식

`_sf_round_challenge_mode2`가 다음 값을 계산함을 증명한다.

$$w = A[y], \quad (\text{S-1})$$

$$w_1 = \text{HighBits}_{512}(w), \quad (\text{S-2})$$

$$c = \text{SampleInBall}(H(w_1, \text{LSB}(\lfloor y_0 \rfloor), \mu), 58). \quad (\text{S-3})$$

`_sf_z_check`가 secret-key NTT 표현과 challenge product를 사용하여

$$z = (z_1, z_2) = y + (-1)^b c s \quad (\text{S-4})$$

를 계산함을 증명한다. 같은 절차가 반환한 $reject = 0$ 은 실제 fixed-point 표현에서 다음 두 조건과 같아야 한다.

$$\|z\|_2^2 \leq 6496508945891328, \quad (\text{S-5})$$

$$\neg(b' = 0 \wedge \|2z - y\|_2^2 < 6505809026482176). \quad (\text{S-6})$$

마지막으로 `_sf_hint_mode2`가

$$h = w_1 - \text{HighBits}_{512}(w - 2\lfloor z_2 \rfloor) \pmod{252} \quad (\text{S-7})$$

를 coefficientwise 계산함을 증명한다.

검증 목표 5.1 (Sign core theorem). 유효한 mode-2 SK, 정확한 μ , 그리고 한 sampler witness (y, b, b') 에서 시작한 실제 `_sf_round_challenge_mode2`–`_sf_z_check`–`_sf_hint_mode2` 연속 실행이 $reject = 0$ 을 반환하면, 그 실행의 (w, w_1, c, z, h) 가 (S-1)–(S-7)을 모두 만족함을 증명한다.

이 목표는 “Sign이 1474 byte를 썼다”보다 강하고, “Sign 분포가 논문 분포와 같다”보다 좁다. 실제 HBZ/rANS wrapper inverse는 z_1 high-part의 byte 표현을 보조하지만, 전체 signature byte theorem에는 아직 h codec과 canonical metadata/padding 정리가 추가로 필요하다.

6 Verify: 복원식과 최종 승인 predicate

검증 당위성. Verify는 공격자가 선택할 수 있는 signature와 message를 받아 승인 여부를 결정하는 외부 신뢰 경계이다. 복원식의 parity, arithmetic shift, centered reduction 또는 hash transcript가 틀리면 honest signature를 거부할 수 있고, boolean 판정이 틀리면 명세의 norm·challenge 조건 밖 object를 승인할 수 있다. 후자가 곧 EUF-CMA 위조를 증명하는 것은 아니지만, 구현에 명세의 보안 정리를 적용할 전제를 없앴다. 그러므로 단순히 “정상 입력을 승인한다”는 충분조건만 증명하지 않고, actual reject = 0과 명세 predicate의 필요충분 동치를 증명해야 한다. parser 성공을 전제로 하는 현재 경계도 정리문에 공개하여 malformed-input 검증과 혼동하지 않는다.

6.1 파싱 이후의 signature object

이 장의 주 정리는 `_unpack_sig_full`이 성공하여 canonical object (x, v, h, c) 를 얻은 경계에서 시작한다. x 는 z_1 의 1024개 high coefficient, v 는 그 1024개 low byte, h 는 512개 hint coefficient, c 는 weight 58 challenge이다.

`_verify_prepare_z1_wprime`가 다음을 계산함을 증명한다.

$$\tilde{z}_1 = 256 \text{ Decode}(x) + v, \quad (\text{V-1})$$

$$w' = \text{LSB}(\tilde{z}_0 - c). \quad (\text{V-2})$$

검증키 $(seed_A, b_1)$ 에서 생성되는 네 열의 행렬을

$$A_1 = (2(a - 2b_1) + qj \mid 2A_{\text{gen}}) \quad (\text{V-3})$$

로 둔다. `_verify_matrix_crt`와 `_sign_verify_recover_w_z2`가 다음 값을 계산함을 증명한다.

$$u = A_1 \tilde{z}_1 - qcj, \quad (\text{V-4})$$

$$w_1 = h + \text{HighBits}_{512}(u) \pmod{252}, \quad (\text{V-5})$$

$$\tilde{z}_2 = \text{center}_q \left(\frac{512w_1 + w'j - u}{2} \right). \quad (\text{V-6})$$

(V-6)의 분자는 coefficientwise 짝수여야 하며, 실제 arithmetic shift와 centered reduction이 위식과 같다는 word/integer bridge를 포함한다.

6.2 승인 조건

실제 mode-2 호출은 norm bound 163265017과 challenge weight 58을 사용한다. `_sign_verify_norm_reject`와 `_sign_verify_tail_m23`에 대해 다음 동치가 목표이다.

$$\begin{aligned} \text{reject} = 0 \iff & \|\tilde{z}_1\|_2^2 + \|\tilde{z}_2\|_2^2 \leq 163265017 \\ & \wedge c = \text{SampleInBall}(H(w_1, w', \tilde{\mu}), 58), \end{aligned} \quad (\text{V-7})$$

여기서

$$\tilde{\mu} = H_{\text{gen}}(vk[0..992], pre, M) \quad (\text{V-8})$$

는 실제 Verify descriptor와 memory가 지정한 byte transcript이다.

검증 목표 6.1 (Verify core theorem). $siglen = 1474$, mode-2 검증키, canonical parsed signature object 및 정확한 descriptor/message memory 계약 아래, 실제 `_verify_prepare_z1_wprime`부터 `_sign_verify_tail_m23`까지의 bounded 실행이 (V-1)–(V-8)을 만족하고, 반환값 $reject = 0$ 이 (V-7)의 두 수학적 조건과 동치임을 증명한다.

전체 `_verify_full_m23`에 대한 임의 malformed byte 입력의 동치는 별도 목표이다. 이 장은 parser 성공을 전제로 숨기지 않고 theorem precondition에 명시한다.

7 KeyGen–Sign–Verify 제한 합성

검증 당위성. KeyGen, Sign, Verify의 국소 정리가 각각 참이어도 서로 다른 key 해석, message transcript 또는 coefficient 표현을 전제로 한다면 end-to-end correctness는 따라오지 않는다. 합성 정리는 실제 호출 사이에서 그 전제들이 동시에 성립하고, KeyGen의 키 관계와 Sign의 hint가 Verify 복원식에 정확히 소비됨을 확인한다. 따라서 이 정리는 개별 산술 정리의 단순 묶음이 아니라, accepted Sign 결과가 actual Verify core에서 승인된다는 구현 수준 completeness를 얻는 마지막 필수 단계이다.

7.1 합성할 세 사실

KeyGen과 Sign 결과가 Verify 계산으로 연결되기 위해 다음 세 등식이 실제 배열 표현에서도 성립해야 한다 [2].

1. Sign의 w_1 과 Verify가 hint를 사용해 복원한 w_1 이 같다.

2.

$$w'j = \text{LSB}(\lfloor y_0 \rfloor)j = \text{LSB}(w) = \text{LSB}(w - 2\lfloor z_2 \rfloor).$$

3.

$$2\lfloor z_2 \rfloor - 2\tilde{z}_2 = \text{LowBits}_{512}(w) - \text{LSB}(w).$$

이 세 식으로 Sign이 hash한 $(w_1, \text{LSB}(\lfloor y_0 \rfloor), \mu)$ 와 Verify가 hash한 $(w_1, w', \tilde{\mu})$ 가 같음을 얻는다. KeyGen의 $As = qj \pmod{2q}$ 는 (V-4)–(V-6)의 재구성식에 사용된다.

검증 목표 7.1 (제한된 implementation completeness). 다음 전제를 둔다.

1. 종료한 accepted KeyGen iteration이 [section 4](#)의 키를 만들었다.

2. 종료한 accepted Sign iteration이 [section 5](#)의 (z, h, c) 를 만들었다.

3. Sign과 Verify가 읽는 VK, prefix/context, message byte가 같아서 $\mu = \tilde{\mu}$ 이다.

4. Verify에 공급한 decoded signature object의 (x, v, h, c) 가 Sign의 $(\lfloor z_1 \rfloor, h, c)$ 와 동일한 수학 object를 나타낸다.

그러면 [section 6](#)의 actual Verify core가

$$reject = 0$$

을 반환함을 증명한다.

이를 논리식으로 쓰면 다음과 같다.

$$\begin{aligned} & \text{KeyGenCoreAccept} \wedge \text{SignCoreAccept} \\ & \wedge \text{DecodedSignatureObjectIdentity} \\ & \implies \text{VerifyCoreReject} = 0. \end{aligned}$$

범위 경계 7.2 (이 합성이 의미하지 않는 것). 이 정리는 KeyGen/Sign의 확률분포 동등성이나 rejection loop의 종료확률을 말하지 않는다. 또한 h 를 포함한 full 1474-byte pack/unpack이 닫히기 전에는 public API byte 배열에 대한 무조건 end-to-end theorem으로 부르지 않는다. 정확한 명칭은 “accepted core execution에 대한 제한된 implementation completeness”이다.

8 아티팩트 산출물과 완료 판정

8.1 작업 순서

이 장은 section 2의 증명 절차를 실제 산출물과 완료 판정으로 구체화한다. 아래 순서는 7일 마감에 맞춘 **GO-MINCORE-7D** 실행 순서이면서 theorem dependency 순서이다. Week 15의 fixed all-6 정리는 종료한 실행의 실패를 배제하는 Hoare 부분정확성이지 termination/losslessness나 non-vacuous 실행 증명이 아니다. 이 미완료 항목과 h byte codec은 이번 일정에서 보류한다. 대신 KeyGen이 제공하는 키 관계, Sign의 accepted core 계산, Verify의 bounded core predicate를 증명하고 decoded signature object 경계에서 합성한다. 앞 단계가 닫히지 않으면 후속 headline theorem에 결론을 새 가정으로 숨기지 않고 정확한 focused actual 경계에서 미완료로 남긴다.

단계	산출물	단아야 할 내용
1일차	범위와 baseline 동결	actual procedure, mode-2 상수, theorem signature와 현재 aggregate fresh baseline을 고정한다.
2일차	KeyGen headline theorem	actual matrix/finalization 경계에서 (KG-1)–(KG-4)와 $As = qj \pmod{2q}$ 를 도출한다. accepted parent lift는 기존 caller 사실로 즉시 합성되는 경우에만 포함한다.
3–4일차	Sign headline theorem	(S-1)–(S-7)과 실제 두 norm predicate를 세 core procedure의 순차 호출로 합성한다.
5–6일차	Verify 및 제한 합성	(V-1)–(V-8)과 $reject = 0$ 동치를 bounded actual core 경계에서 증명하고, 동일 key/message transcript와 decoded-object identity 아래 accepted Sign 결과가 Verify core에서 승인됨을 증명한다.
7일차	Paper artifact	source hash, extraction identity, safety evidence, theorem manifest, fresh -no-eco build, proof-hole/axiom scan, 독립 audit와 TCB를 제공한다.

8.2 각 headline theorem의 완료 기준

완료 기준 8.1 (기계검증 완료). KeyGen, Sign, Verify 정리는 각각 다음 조건을 모두 만족할 때만 완료로 판정한다.

1. 최종 정리가 pinned extraction의 실제 프로시저를 직접 호출한다.
2. 성공, decoded equality, trace equality 또는 최종 predicate를 전제로 넣지 않고 실제 하위 postcondition에서 도출한다.
3. 모든 representation, bounds, frame 및 memory-disjointness 전제를 theorem statement에 공개한다.
4. Hoare partial correctness, losslessness, termination을 구분하여 표기한다.
5. 신규 타깃 개별 fresh compile과 aggregate fresh -no-eco compile이 모두 통과한다.
6. proof hole, authored axiom, debug declaration, extraction/table drift가 없다.

7. claim ledger, theorem graph, proof manifest와 논문의 상태가 일치한다.

8.3 안전성 및 TCB

Jasmin compiler와 EasyCrypt extraction의 의미 연결은 safe program에 대해 적용된다 [5]. 따라서 assembly-level correctness를 주장하려면 논문 대상 프로시저의 memory safety 증거를 함께 제공한다. 이를 제공하지 못한 프로시저는 “extracted EasyCrypt model 수준”의 결과로만 표현한다. TCB에는 적어도 다음을 명시한다.

- pinned HAETAE 명세와 그 EasyCrypt formalization,
- Jasmin instruction semantics, compiler proof 및 jasmin2ec,
- EasyCrypt kernel/SMT backends와 사용한 버전,
- generated mode-2 tables와 extraction scripts,
- theorem statement와 구현 배열을 수학 object로 해석하는 bridge.

8.4 명시적 비주장

최종 핵심 연산 논문은 다음을 주장하지 않는다.

- KeyGen sampler와 hyperball sampler의 정확한 출력분포,
- KeyGen/Sign rejection loop의 일반 termination 또는 기대 반복 횟수,
- 모든 canonical HBZ stream의 rANS encoder 성공,
- h codec, metadata, padding을 포함한 full 1474-byte canonical parser,
- 임의 malformed signature의 완전한 rejection characterization,
- 전체 public API의 분포 동등성,
- HAETAE 구현의 EUF-CMA 또는 논문 보안 환원 전체.

8.5 예상 작업량과 보고 제목

현재 추출, memory bridge, raw μ transcript, signature prefix 및 HBZ/rANS vertical slice를 재사용한다 [3, 4]. 활성 산출물의 시간 한도는 7일이다. 이는 주장 강도를 높이는 근거가 아니라 범위를 decoded-object MINCORE로 동결하는 제약이다. upper caller lift가 일정에 맞지 않으면 관찰용 모델로 대체하지 않고 focused actual core 경계를 명시한다. full byte codec, public API, termination과 분포까지 포함하는 기존 장기 계획은 이번 산출물 범위 밖이다.

최종 보고 제목은 다음과 같이 고정한다.

**HAETAE-2 Jasmin 구현의 KeyGen 키 방정식,
Sign 응답·힌트 계산 및 Verify 재구성·승인 계산에 대한 기계검증**

“HAETAE 전체 기능정확성 검증” 또는 “HAETAE 구현 보안 증명”이라는 제목은 위 비주장들이 닫히기 전에는 사용하지 않는다.

참고문헌

- [1] CryptoLab. Pinned HAETAE jasmin implementation tree, 2026. Local source root haetae-ref-jasmin/jasmin.
- [2] HAETAE Team. HAETAE specification document. NIST Additional Post-Quantum Digital Signature Schemes, 2023. Algorithm and parameter reference for KeyGen, Sign, and Verify.
- [3] HAETAE top-down EasyCrypt project. Claim ledger, 2026. Local source of truth: haetae-topdown-easycrypt/CLAIM_LEDGER.md.
- [4] HAETAE top-down EasyCrypt project. Week 14 report: Production full HBZ wrapper boundary, 2026. Local report: haetae-topdown-easycrypt/WEEK14_REPORT.md.
- [5] Jasmin Project. Easycrypt extraction, 2026. The extraction correctness boundary is stated for safe programs.